NetSentinel: Real-Time Network Security Monitoring & IDS

12-Week Project Plan (6 July – 26 September 2026)

Java PBL A.Y. 2026-27 | Network Security & Systems Architecture Track

Team Members

Name	Roll No.	Module
Nandlal Upman	24E1ARADM40P108	Packet Capture & Preprocessing (Pure Java PCAP & Decoys)
Piyush Agarwal	24E1ARADM40P120	Rule-Based Detection Engine (Sliding-Window ArrayDeque)
Mayank Jangid	24E1ARADM30P097	Anomaly & ML Detection Engine (Shannon Entropy & Flows)
Nikhil Kumar (Team Lead)	24E1ARADM30P110	Central Backend & SQLite Storage (Spring Boot 3 + JDBC)
Parikshit Vyas	24E1ARADM40P119	SOC Web Dashboard & Frontend (Chart.js & Telemetry UI)

Saturday = Mentor Meeting (All members attend) | Sunday = Review / Self-Study / Code Inspection

Project Architecture & Execution Environment

Platform: NetSentinel (Shield NSM Platform) is a zero-dependency, pure Java network intrusion detection framework engineered on Java 21 LTS, Spring Boot 3, and Project Loom Virtual Threads.

Core Modules: Five decoupled components communicating via frozen JSON contracts: Module 1 Packet Dissection → Module 2 Deterministic Rule Engine → Module 3 Flow Anomaly Inference → Module 4 SQLite Storage Service → Module 5 Live SOC Web Dashboard.

Duration & Schedule: 12 Academic Weeks (6 July – 26 September 2026), 72 operating days, concluding with final project submission and viva defense.

Project Guide: Er. Ram Babu Buri (Assistant Professor, Department of Computer Science & IT).

NANDLAL UPMAN — 24E1ARADM40P108

Module: Packet Capture & Preprocessing (Person 1)

Day	Date	My Work for the Day
WEEK 1 — Team Formation + Abstract		
Mon	6 Jul	Joined the NetSentinel team. Registered team on portal under guide Er. Ram Babu Buri. Agreed on building pure Java packet capture without native WinPcap/Npcap drivers.
Tue	7 Jul	Studied libpcap binary file format specifications: 24-byte global file header and 16-byte packet record headers. Noted required fields.
Wed	8 Jul	Researched unprivileged socket listening in Java. Discovered decoy listener concept for ports 2222, 8088, 8888, and 9090 to trap attacks over physical LAN.
Thu	9 Jul	Drafted the Packet Capture and Preprocessing section for the abstract (about 250 words) and submitted to team lead Nikhil Kumar.
Fri	10 Jul	Created module-1-packet-capture folder structure with Maven pom.xml and did initial commit to GitHub repository.
SAT	**11 Jul**	**Mentor meeting. Presented packet capture architecture without C/C++ drivers. Mentor approved pure Java libpcap parser and decoy sockets.**
Sun	12 Jul	Reviewed mentor feedback. Listed protocol header offsets for Ethernet (14 bytes), IPv4 (20 bytes), and TCP (20 bytes).
WEEK 2 — SRS Documentation		
Mon	13 Jul	Wrote SRS Section 1 for Module 1: functional scope of packet ingestion from binary .pcap captures and live decoy sockets.
Tue	14 Jul	Wrote functional requirements FR-01 (PCAP file parsing) and FR-02 (Decoy honeypot listeners on unprivileged ports 2222, 8088, 8888, 9090).
Wed	15 Jul	Defined non-functional requirements: minimum throughput > 1,000 pkts/sec and packet parsing latency under 0.5 ms.
Thu	16 Jul	Documented use cases: 'Ingest PCAP Stream' and 'Trap Incoming Honeypot Connection' with success and exception flows.
Fri	17 Jul	Combined Module 1 requirements into master SRS document with team. Fixed formatting and pushed updates to GitHub.
SAT	**18 Jul**	**Mentor meeting. Presented Module 1 SRS requirements. Guide approved frozen JSON contract format for emitted packet events.**
Sun	19 Jul	Incorporated mentor corrections regarding snaplen boundary handling in pcap file parsing.
WEEK 3 — UML Design		
Mon	20 Jul	Created UML Use Case diagram for packet capture subsystem using draw.io: parsing, frame dissection, and streaming.
Tue	21 Jul	Designed Class Diagram for Module 1: PcapBinaryParser, LiveDecoyProbeListener, PacketData, and PacketGenerator.
Wed	22 Jul	Constructed Sequence Diagram showing raw byte stream flow into ByteBuffer, frame extraction, and emit to consumer callback.
Thu	23 Jul	Aligned method signatures with Module 2 (RuleEngine) and Module 4 (PipelineCoordinator) interfaces to ensure consistency.
Fri	24 Jul	Exported high-resolution UML diagrams and committed them to GitHub repository.
SAT	**25 Jul**	**Mentor meeting. Presented Module 1 class and sequence diagrams. Mentor approved decoupling packet parser from storage.**
Sun	26 Jul	Refined diagram notes on Project Loom Virtual Thread execution for socket listeners.
WEEK 4 — Architecture, DB Schema & Contracts		
Mon	27 Jul	Authored PacketData.java immutable record: timestamp, src_ip, dst_ip, src_port, dst_port, protocol, flags, length.
Tue	28 Jul	Created factory method PacketData.create() with defensive null handling and default values for ICMP/UDP packets.
Wed	29 Jul	Drafted sample pcap file with Wireshark containing synthetic TCP SYN packets to serve as test fixture.
Thu	30 Jul	Configured Maven pom.xml with Java 21 release flag and JUnit 5 test dependencies.
Fri	31 Jul	Tested compiling Module 1 skeleton with 'mvn clean compile'. Build succeeded with zero warnings.

Day	Date	My Work for the Day
SAT	**1 Aug**	**Mentor meeting. Showed PacketData contract and Maven structure. Mentor verified field schemas match downstream requirements.**
Sun	2 Aug	Planned binary parsing logic for Big-Endian and Little-Endian pcap magic headers.

NANDLAL UPMAN — 24E1ARADM40P108

Module: Packet Capture & Preprocessing (Person 1)

Day	Date	My Work for the Day
WEEK 5 — Module Coding Phase 1		
Mon	3 Aug	Created PcapBinaryParser.java. Implemented magic number detection: 0xA1B2C3D4 (Big-Endian) and 0xD4C3B2A1 (Little-Endian).
Tue	4 Aug	Wrote global header reader in ByteBuffer: parsed major version, minor version, thiszone, sigfigs, and snaplen fields.
Wed	5 Aug	Implemented packet record loop extracting ts_sec, ts_usec, incl_len, and orig_len. Added guard against corrupted lengths.
Thu	6 Aug	Handled timestamp generation converting epoch seconds and microseconds into java.time.Instant with nanosecond precision.
Fri	7 Aug	Tested reading a 100-packet capture dump. Verified correct frame slicing and stream iteration.
SAT	**8 Aug**	**Mentor meeting. Demonstrated raw pcap file reader outputting packet record timestamps. Mentor approved binary parser core.**
Sun	9 Aug	Researched Ethernet frame slicing and EtherType offsets for IPv4 (0x0800).
WEEK 6 — Module Coding Phase 2		
Mon	10 Aug	Developed dissectFrame() method: validated Ethernet header (14 bytes) and isolated IPv4 payload.
Tue	11 Aug	Extracted IPv4 header length (IHL), Total Length, Protocol number (6 for TCP, 17 for UDP, 1 for ICMP), and IP addresses.
Wed	12 Aug	Wrote formatIp() helper converting raw 4-byte arrays into standard dotted-decimal strings (e.g. 192.168.1.1).
Thu	13 Aug	Implemented TCP header parser: extracted source port, destination port, sequence number, and acknowledgment number.
Fri	14 Aug	Implemented bitwise flag extraction for TCP flags: SYN, ACK, FIN, RST, PSH, and URG via formatTcpFlags() method.
SAT	**15 Aug**	**Mentor meeting. Showed full packet dissection outputting structured PacketData. Mentor validated accuracy of TCP flag decoding.**
Sun	16 Aug	Added support for UDP and ICMP packet extraction to provide baseline background traffic parsing.
WEEK 7 — Module Coding Phase 3		
Mon	17 Aug	Created LiveDecoyProbeListener.java. Implemented multi-port server socket listener on decoy ports 2222, 8088, 8888, 9090.
Tue	18 Aug	Integrated Java 21 Project Loom Virtual Threads (Thread.ofVirtual()) to handle incoming TCP probe connections asynchronously.
Wed	19 Aug	Configured socket listener to extract remote client IP and port upon connection before terminating the socket cleanly.
Thu	20 Aug	Synthesized PacketData records from live socket triggers and dispatched them directly into the downstream detection pipeline.
Fri	21 Aug	Tested live decoy probe listener by connecting via PowerShell Test-NetConnection on port 2222. Confirmed immediate packet capture.
SAT	**22 Aug**	**Mentor meeting. Demonstrated live decoy probe trapping connection on port 2222 without root permissions. Mentor signed off Phase 3.**
Sun	23 Aug	Planned synthetic attack packet generator for port scan, SYN flood, and brute force scenarios.
WEEK 8 — Module Coding Phase 4: Final		
Mon	24 Aug	Created PacketGenerator.java: implemented generatePortScan() creating multi-port SYN packets across port range.
Tue	25 Aug	Implemented generateSynFlood() producing rapid high-volume SYN packets targeting a single victim port.
Wed	26 Aug	Implemented generateBruteForce() generating repeated connection attempts against SSH port 22.
Thu	27 Aug	Wrote PacketStreamEmitter.java supporting JSON-lines (.jsonl) serialization and HTTP POST ingestion to Module 4.
Fri	28 Aug	Cleaned Java 21 compiler warnings on unused variables 'network' and 'origLen' in PcapBinaryParser using in-place buffer cursor positioning. Zero-warning build.

Day	Date	My Work for the Day
SAT	**29 Aug**	**Mentor meeting. Showed all generator attack scenarios and zero-warning build. Mentor confirmed Module 1 is complete and ready for integration.**
Sun	30 Aug	Wrote unit tests in PacketCaptureTest.java verifying pcap dissection, flag parsing, and attack generators.

NANDLAL UPMAN — 24E1ARADM40P108

Module: Packet Capture & Preprocessing (Person 1)

Day	Date	My Work for the Day
WEEK 9 — Integration + Testing		
Mon	31 Aug	Connected Module 1 with Piyush's RuleEngine and Mayank's AnomalyEngine inside Nikhil's NsmPipelineCoordinator.
Tue	1 Sep	Tested continuous streaming of 500 synthetic packets. Confirmed zero packet loss across Loom virtual thread dispatch.
Wed	2 Sep	Tested pcap replay endpoint (/api/attack/pcap-upload). Verified 1,000 packets replayed at 15ms intervals cleanly.
Thu	3 Sep	Verified decoy sockets bind 0.0.0.0 without colliding with Spring Boot port 8080.
Fri	4 Sep	Executed full integration test suite: 'mvn test -pl module-1-packet-capture'. All tests passed with 100% success.
SAT	**5 Sep**	**Mentor meeting. Demonstrated full pipeline integration: PCAP feed flowed through rules and anomaly engine into SQLite. Mentor approved.**
Sun	6 Sep	Prepared multi-host test scripts for live physical Wi-Fi/LAN testing.
WEEK 10 — Multi-Host Deployment & Network Simulation		
Mon	7 Sep	Deployed NetSentinel on primary laptop over physical university Wi-Fi subnet with active decoy listeners on ports 2222, 8088, 8888, 9090.
Tue	8 Sep	Executed live external probes from secondary host: 'curl http://192.168.1.5:8088'. Decoy honeypot captured incoming TCP handshakes instantly via virtual threads.
Wed	9 Sep	Identified that localhost test probes emit IPv6 loopback addresses (0:0:0:0:0:0:0:1 / ::1), disrupting downstream IP geolocation and rule matching.
Thu	10 Sep	Implemented IPv6 Loopback Normalization in LiveDecoyProbeListener.java, converting 0:0:0:0:0:0:0:1 and ::1 into standard IPv4 127.0.0.1.
Fri	11 Sep	Tested rapid curl bursts against decoy HTTP port 8088 and SSH port 2222. Verified correct TCP socket closure and clean packet dispatch into pipeline.
SAT	**12 Sep**	**Mentor meeting. Demonstrated live physical LAN decoy trapping and normalized IPv4 telemetry from external machines. Mentor approved Week 10.**
Sun	13 Sep	Reviewed socket thread allocation metrics; confirmed virtual threads maintained sub-1% CPU utilization during multi-probe bursts.
WEEK 11 — Performance Profiling, Report + PPT + Video		
Mon	14 Sep	Benchmarked binary packet dissection throughput: sustained 1,450 pkts/sec throughput with mean parsing latency of 0.08 ms per packet.
Tue	15 Sep	Audited Maven reactor build; verified zero compiler warnings across all Module 1 classes under strict Java 21 compiler flags (-Xlint:all).
Wed	16 Sep	Authored Module 1 technical documentation chapter: libpcap file specification, virtual-thread socket architecture, and decoy trap mechanics.
Thu	17 Sep	Prepared technical presentation slides covering pure Java binary parsing without native WinPcap/Npcap C drivers.
Fri	18 Sep	Recorded technical demonstration video clip highlighting live decoy socket trapping and PCAP file upload. Sent clip to lead Nikhil Kumar.
SAT	**19 Sep**	**Mentor meeting. Presented Module 1 documentation chapter, benchmark tables, and presentation slides. Mentor approved Week 11.**
Sun	20 Sep	Incorporated mentor feedback regarding libpcap snaplen boundary handling in documentation.
WEEK 12 — Final Submission & Project Viva Completion		
Mon	21 Sep	Verified standalone PCAP upload and decoy honeypot listeners inside packaged executable Fat JAR (module-4-backend-storage-1.0.0.jar).
Tue	22 Sep	Validated dual-repo mirror synchronization and SHA-256 byte-for-byte integrity for all Module 1 source files and test fixtures.
Wed	23 Sep	Rehearsed viva technical explanations: endianness byte flipping, TCP 3-way handshake dissection, and Virtual Thread concurrency advantages.

Day	Date	My Work for the Day
Thu	24 Sep	Participated in full team mock viva defense and end-to-end live intrusion demonstration rehearsal.
Fri	25 Sep	Conducted final code and documentation audit; verified all Git tags and academic deliverables.
SAT	**26 Sep**	**Mentor meeting. Final Project Defense before Project Guide Er. Ram Babu Buri. Received outstanding evaluation marks; Module 1 officially signed off!**
Sun	27 Sep	Academic project archive verification: confirmed final grade records, repository tags, and signed documentation archived.

PIYUSH AGARWAL — 24E1ARADM40P120

Module: Rule-Based Detection Engine (Person 2)

Day	Date	My Work for the Day
WEEK 1 — Team Formation + Abstract		
Mon	6 Jul	Joined NetSentinel team. Selected Module 2 (Rule-Based Detection Engine). Agreed on deterministic sliding-window architecture.
Tue	7 Jul	Studied sliding-window time queue algorithms for intrusion detection using in-memory Java collections.
Wed	8 Jul	Researched signature heuristics for horizontal port scans, TCP SYN floods, and brute force login attempts.
Thu	9 Jul	Wrote Module 2 scope for the project abstract: sliding time queues, threshold rules, and cooldown deduplication.
Fri	10 Jul	Created module-2-rule-engine directory, initialized Maven pom.xml, and performed initial commit to GitHub.
SAT	**11 Jul**	**Mentor meeting. Presented rule-based detection approach. Mentor advised incorporating alert cooldown to prevent notification fatigue.**
Sun	12 Jul	Reviewed mentor feedback. Designed sliding time-window queue data structures using ArrayDeque.
WEEK 2 — SRS Documentation		
Mon	13 Jul	Wrote functional requirements for port scan rule: trigger alert if unique destination ports >= 15 within 5-second window.
Tue	14 Jul	Wrote requirements for SYN flood rule: trigger alert if SYN packet count >= 50 within 2-second window.
Wed	15 Jul	Wrote requirements for brute force rule: trigger alert if connection attempts to port 22/3389 exceed 10 within 30 seconds.
Thu	16 Jul	Documented AlertData output contract with fields: alert_id, timestamp, attack_type, severity, src_ip, dst_ip, confidence.
Fri	17 Jul	Integrated Module 2 requirements into team SRS document and pushed to GitHub repository.
SAT	**18 Jul**	**Mentor meeting. Presented detection threshold parameters. Mentor approved sliding-window mathematical boundaries.**
Sun	19 Jul	Formulated time-window eviction policy to guarantee constant O(1) memory consumption.
WEEK 3 — UML Design		
Mon	20 Jul	Created UML Use Case diagram for RuleEngine evaluating packets and generating security alerts.
Tue	21 Jul	Designed Class Diagram: RuleEngine, AlertData, sliding-window queue structures, and cooldown managers.
Wed	22 Jul	Constructed State Machine diagram modeling transition from Normal to Attack Detected and Cooldown state.
Thu	23 Jul	Created Sequence Diagram showing PacketData arriving, queue updates, threshold evaluation, and AlertData emission.
Fri	24 Jul	Exported all UML diagrams to high-resolution PNGs and pushed them to GitHub repository.
SAT	**25 Jul**	**Mentor meeting. Showed state machine and sequence diagrams. Mentor approved alert deduplication logic.**
Sun	26 Jul	Reviewed thread-safety requirements for concurrent packet evaluation.
WEEK 4 — Architecture, DB Schema & Contracts		
Mon	27 Jul	Authored AlertData.java record matching frozen contract: alertId, timestamp, attackType, severity, srcIp, dstIp, details, confidence, detectionMethod.
Tue	28 Jul	Created factory method AlertData.createRuleAlert() setting detection_method to 'rule' automatically.
Wed	29 Jul	Set up inter-module dependency in pom.xml to import module-1-packet-capture for PacketData access.
Thu	30 Jul	Implemented skeleton RuleEngine.java with evaluatePacket(PacketData packet) method.
Fri	31 Jul	Tested compiling module-2-rule-engine against module-1-packet-capture dependency. Compilation passed.
SAT	**1 Aug**	**Mentor meeting. Showed inter-module POM dependency and AlertData contract. Mentor verified schema alignment with backend.**
Sun	2 Aug	Planned port scan sliding-window map structures using ConcurrentHashMap.

PIYUSH AGARWAL — 24E1ARADM40P120

Module: Rule-Based Detection Engine (Person 2)

Day	Date	My Work for the Day
WEEK 5 — Module Coding Phase 1		
Mon	3 Aug	Implemented port scan sliding window: ConcurrentHashMap> tracking probed destination ports per source IP.
Tue	4 Aug	Wrote window eviction logic: automatically remove port timestamps older than 5 seconds (PORT_SCAN_WINDOW_SEC).
Wed	5 Aug	Implemented threshold check: if unique port count >= 15, generate HIGH severity port_scan alert.
Thu	6 Aug	Wrote automated test simulating 20 packets across ports 20-39 from 192.168.1.50. Port scan alert fired successfully.
Fri	7 Aug	Added unit test verifying benign traffic touching only port 80 and 443 does not trigger false positive port scan alert.
SAT	**8 Aug**	**Mentor meeting. Demonstrated port scan detection rule with synthetic packets. Mentor verified zero false positives on benign web traffic.**
Sun	9 Aug	Designed SYN flood sliding-window queue using ArrayDeque for frequency rate evaluation.
WEEK 6 — Module Coding Phase 2		
Mon	10 Aug	Implemented SYN flood sliding window: ConcurrentHashMap> tracking SYN arrivals per victim port.
Tue	11 Aug	Wrote SYN arrival rate eviction: prune timestamps older than 2 seconds (SYN_FLOOD_WINDOW_SEC).
Wed	12 Aug	Added SYN threshold check: if queue size >= 50, trigger CRITICAL severity syn_flood alert with confidence 0.95.
Thu	13 Aug	Tested SYN flood detector against burst of 60 SYN packets within 500ms. Alert fired correctly on packet 50.
Fri	14 Aug	Validated that non-SYN packets (ACK, FIN, UDP) are ignored by the SYN flood detector without polluting queues.
SAT	**15 Aug**	**Mentor meeting. Showed SYN flood rate tracking live. Mentor validated rate threshold against burst conditions.**
Sun	16 Aug	Planned brute force detector targeting sensitive authentication ports (21, 22, 23, 3389, 8080).
WEEK 7 — Module Coding Phase 3		
Mon	17 Aug	Implemented brute force detection rule: track connection attempts to sensitive admin ports within 30-second window.
Tue	18 Aug	Added threshold check: if connection attempts >= 10 to same target port, fire MEDIUM severity brute_force alert.
Wed	19 Aug	Tested brute force detection with 12 synthetic SSH connection attempts to port 22 from 192.168.1.77. Alert fired on attempt 10.
Thu	20 Aug	Built Alert Deduplication and Cooldown Engine: ConcurrentHashMap tracking cooldown per (src_ip, attack_type).
Fri	21 Aug	Configured 5-second cooldown interval: suppressed duplicate alerts during sustained attack sweeps to eliminate alert fatigue.
SAT	**22 Aug**	**Mentor meeting. Demonstrated brute force detection and alert cooldown suppressing 980 redundant alerts during 1000-packet sweep. Mentor approved.**
Sun	23 Aug	Refactored queue cleanup routines to ensure memory boundedness during long-running execution.
WEEK 8 — Module Coding Phase 4: Final		
Mon	24 Aug	Added reset() method to RuleEngine clearing all tracking maps, queues, and cooldown states for instant demo resets.
Tue	25 Aug	Fine-tuned confidence scoring: 0.95 for SYN flood, 0.90 for port scan, and 0.85 for brute force attacks.
Wed	26 Aug	Benchmarked evaluation latency per packet: achieved mean latency of 0.12 ms, well within sub-millisecond requirement.
Thu	27 Aug	Created RuleEngineTest.java with 4 comprehensive test cases covering port scan, SYN flood, brute force, and cooldown suppression.
Fri	28 Aug	Executed 'mvn clean test'. All 4 unit tests passed in 0.05 seconds. Pushed final Module 2 code to GitHub repository.
SAT	**29 Aug**	**Mentor meeting. Demonstrated all unit test cases passing. Mentor signed off Module 2 as complete.**
Sun	30 Aug	Prepared integration test plan with Module 1 packet emitter and Module 4 backend.

PIYUSH AGARWAL — 24E1ARADM40P120

Module: Rule-Based Detection Engine (Person 2)

Day	Date	My Work for the Day
WEEK 9 — Integration + Testing		
Mon	31 Aug	Integrated RuleEngine into Nikhil's NsmPipelineCoordinator. Verified incoming PacketData records trigger alert callbacks.
Tue	1 Sep	Tested concurrent packet evaluation: verified RuleEngine and AnomalyEngine process packets simultaneously without thread locking.
Wed	2 Sep	Validated reset endpoint (/api/alerts/clear): confirmed coordinator resets RuleEngine state and clears active cooldowns.
Thu	3 Sep	Tested continuous 1,000 packet ingestion: verified 100% detection rate on multi-port sweeps with zero duplicate alerts.
Fri	4 Sep	Verified generated AlertData records successfully persist into SQLite database via AlertRepository.
SAT	**5 Sep**	**Mentor meeting. Demonstrated integrated pipeline with RuleEngine alerts showing in SQLite and on frontend dashboard. Mentor approved.**
Sun	6 Sep	Configured multi-host test scenarios with Nmap, hping3, and Hydra.
WEEK 10 — Multi-Host Deployment & Network Simulation		
Mon	7 Sep	Participated in live multi-host network deployment over physical university Wi-Fi subnet.
Tue	8 Sep	Tested rule engine response against external curl and nmap probes directed at decoy ports 2222 and 8088.
Wed	9 Sep	Verified honeypot trap rule correctly flagged external LAN probe as CRITICAL intrusion with confidence 0.95.
Thu	10 Sep	Analyzed TCP connection retries from external clients; tuned cooldown window to prevent alert log spam while ensuring re-probes trigger alerts.
Fri	11 Sep	Validated seamless handling of normalized IPv4 addresses (127.0.0.1) without breaking source IP tracking hash maps.
SAT	**12 Sep**	**Mentor meeting. Demonstrated live physical LAN attack detection with 100% accuracy and zero false positives. Mentor approved Week 10.**
Sun	13 Sep	Compiled detection latency benchmarks: mean rule evaluation latency measured at 0.12 ms per packet.
WEEK 11 — Performance Profiling, Report + PPT + Video		
Mon	14 Sep	Profiled memory footprint under sustained 6-hour burn-in load; verified memory bounded under 25MB for rule tracking maps.
Tue	15 Sep	Collaborated with Parikshit to map rule signatures to enterprise MITRE ATTACK techniques (T1046 Scanning, T1499 DoS, T1110 Brute Force).
Wed	16 Sep	Authored Module 2 report chapter: sliding time-window queue mathematics, deterministic eviction algorithms, and deduplication logic.
Thu	17 Sep	Prepared presentation slides covering time complexity analysis (O(1) amortized eviction) and rule benchmarks.
Fri	18 Sep	Recorded video demonstration showing real-time sliding-window rate tracking during live SYN flood simulation. Sent to lead Nikhil Kumar.
SAT	**19 Sep**	**Mentor meeting. Submitted Module 2 report draft and presentation slides. Mentor verified detection accuracy tables.**
Sun	20 Sep	Polished report chapter based on mentor feedback. Checked GitHub commit log.
WEEK 12 — Final Submission & Project Viva Completion		
Mon	21 Sep	Validated rule engine execution inside standalone Fat JAR launcher (run.bat) on isolated offline test laptop.
Tue	22 Sep	Audited Module 2 source code and verified 100% SHA-256 match across both mirrored Git repositories.
Wed	23 Sep	Rehearsed viva technical questions on sliding-window memory safety, amortized O(1) deque operations, and false-positive suppression.
Thu	24 Sep	Participated in full team dry-run presentation and live intrusion simulation demonstration.
Fri	25 Sep	Performed final documentation check and confirmed all Module 2 test deliverables in Git repository.
SAT	**26 Sep**	**Mentor meeting. Successfully defended Module 2 before Project Guide Er. Ram Babu Buri. Received highest grade; Module 2 officially signed off!**

Day	Date	My Work for the Day
Sun	**27 Sep**	Academic project archive verification: confirmed final grade records, repository tags, and signed documentation archived.

MAYANK JANGID — 24E1ARADM30P097

Module: Anomaly & ML Detection Engine (Person 3)

Day	Date	My Work for the Day
WEEK 1 — Team Formation + Abstract		
Mon	6 Jul	Joined NetSentinel team. Assigned Module 3 (Anomaly & ML Detection Engine). Focused on behavioral statistical modeling.
Tue	7 Jul	Researched statistical anomaly detection techniques in network traffic: Shannon entropy, flow feature aggregation, and SYN-to-ACK ratios.
Wed	8 Jul	Studied Shannon Port Entropy formula: H = -sum(p_i * log2(p_i)) to distinguish stealthy port sweeps from benign traffic.
Thu	9 Jul	Drafted Anomaly & ML Detection section for abstract (about 250 words) and submitted to lead Nikhil Kumar.
Fri	10 Jul	Created module-3-anomaly-detection folder with Maven pom.xml and did initial commit to GitHub repository.
SAT	**11 Jul**	**Mentor meeting. Presented Shannon entropy and flow modeling concept. Mentor approved pure Java mathematical implementation.**
Sun	12 Jul	Reviewed mentor feedback. Listed 6 flow statistical features to extract from packet streams.
WEEK 2 — SRS Documentation		
Mon	13 Jul	Wrote SRS functional requirements for flow aggregation: aggregate packets into bidirectional flows over 5-second windows.
Tue	14 Jul	Wrote requirements for Shannon port entropy: calculate dynamic entropy across destination ports and flag H > 2.5.
Wed	15 Jul	Wrote requirements for SYN-to-ACK imbalance ratio: detect half-open scanning when SYN count >> ACK count.
Thu	16 Jul	Documented AlertData tagging: all anomaly engine alerts must carry 'detection_method': 'ml' and dynamic confidence score.
Fri	17 Jul	Integrated Module 3 requirements into team SRS document and pushed updates to GitHub repository.
SAT	**18 Jul**	**Mentor meeting. Showed Module 3 SRS specifications. Guide approved mathematical baseline thresholds.**
Sun	19 Jul	Planned flow feature extraction data structures in Java using ConcurrentHashMap.
WEEK 3 — UML Design		
Mon	20 Jul	Created UML Use Case diagram for flow aggregation, feature extraction, and anomaly classification.
Tue	21 Jul	Designed Class Diagram: FlowFeatureExtractor, FlowFeatures, and AnomalyDetectionEngine with inter-module links.
Wed	22 Jul	Created Activity Diagram for flow processing: receive packet, update flow aggregate, calculate entropy, evaluate threshold, emit alert.
Thu	23 Jul	Constructed Sequence Diagram showing flow lifecycle and interaction with NsmPipelineCoordinator.
Fri	24 Jul	Exported UML diagrams to PNG, committed to GitHub, and added to team documentation folder.
SAT	**25 Jul**	**Mentor meeting. Presented flow activity and sequence diagrams. Mentor verified asynchronous flow processing design.**
Sun	26 Jul	Studied efficient log2 mathematical algorithms in Java to minimize CPU overhead.
WEEK 4 — Architecture, DB Schema & Contracts		
Mon	27 Jul	Authored FlowFeatures.java immutable record: flowKey, packetCount, totalBytes, synCount, ackCount, portEntropy, meanInterArrivalMs.
Tue	28 Jul	Configured Maven pom.xml with Java 21 compiler and dependency on module-1-packet-capture for PacketData access.
Wed	29 Jul	Designed flow key structure: (src_ip -> dst_ip) string representation for bidirectional flow hashing.
Thu	30 Jul	Implemented skeleton FlowFeatureExtractor.java and AnomalyDetectionEngine.java.
Fri	31 Jul	Tested compiling Module 3 with 'mvn clean compile'. Build completed with zero errors.
SAT	**1 Aug**	**Mentor meeting. Showed FlowFeatures contract and Maven setup. Mentor approved flow record schema.**

Day	Date	My Work for the Day
Sun	**2 Aug**	Planned flow aggregation accumulator classes for sliding 5-second windows.

MAYANK JANGID — 24E1ARADM30P097

Module: Anomaly & ML Detection Engine (Person 3)

Day	Date	My Work for the Day
WEEK 5 — Module Coding Phase 1		
Mon	3 Aug	Implemented FlowFeatureExtractor.java with inner FlowAggregate accumulator class tracking packet counts and timestamps.
Tue	4 Aug	Added byte volume tracking and TCP flag counters (SYN vs ACK) to the flow accumulator.
Wed	5 Aug	Implemented inter-arrival time calculation: computed rolling difference between consecutive packet timestamps.
Thu	6 Aug	Wrote port frequency tracking map: ConcurrentHashMap recording distribution of destination ports.
Fri	7 Aug	Tested flow feature extractor with 50 synthetic packets. Verified packet count, bytes, and inter-arrival averages calculated accurately.
SAT	**8 Aug**	**Mentor meeting. Demonstrated flow aggregation and statistical feature output. Mentor approved accumulator design.**
Sun	9 Aug	Formulated Shannon Port Entropy calculation logic in Java using Math.log().
WEEK 6 — Module Coding Phase 2		
Mon	10 Aug	Implemented calculatePortEntropy(): computed probability distribution p_i = count / total and summed -p_i * log2(p_i).
Tue	11 Aug	Validated entropy mathematics: confirmed uniform traffic to 1 port yields H = 0.0, while traffic scattered across 32 ports yields H = 5.0.
Wed	12 Aug	Implemented SYN-to-ACK ratio calculation: flagged flows with synCount > 20 and ackCount == 0 as anomalous half-open sweeps.
Thu	13 Aug	Tested entropy against synthetic port scan: entropy crossed 3.2, cleanly exceeding the 2.5 anomaly threshold.
Fri	14 Aug	Tested entropy against benign single-port web browsing: entropy remained below 0.8, generating zero false positives.
SAT	**15 Aug**	**Mentor meeting. Showed live entropy calculation contrast between web traffic (H < 1.0) and port scan (H > 3.0). Mentor approved.**
Sun	16 Aug	Planned classification and confidence scoring logic in AnomalyDetectionEngine.
WEEK 7 — Module Coding Phase 3		
Mon	17 Aug	Created AnomalyDetectionEngine.java. Implemented evaluateFlow(FlowFeatures features) method.
Tue	18 Aug	Implemented anomaly heuristic: if portEntropy > 2.5 and packetCount >= 15, trigger HIGH severity port_scan anomaly alert.
Wed	19 Aug	Implemented SYN imbalance heuristic: if synCount > 40 and synToAckRatio > 10, trigger HIGH severity syn_flood anomaly alert.
Thu	20 Aug	Added dynamic confidence scoring: scaled confidence between 0.70 and 0.99 proportional to entropy distance and packet frequency.
Fri	21 Aug	Ensured emitted AlertData records are tagged with detection_method = 'ml' to enable visual purple badge rendering on dashboard.
SAT	**22 Aug**	**Mentor meeting. Demonstrated AnomalyDetectionEngine emitting 'ml' alerts with dynamic confidence scores. Mentor approved Phase 3.**
Sun	23 Aug	Analyzed memory lifecycle of flow aggregation maps under sustained packet streams.
WEEK 8 — Module Coding Phase 4: Final		
Mon	24 Aug	Implemented flow lifecycle pruning: automatically purge inactive flows older than 10 seconds to prevent heap memory growth.
Tue	25 Aug	Optimized entropy calculations using pre-computed log2 mathematical approximations, achieving a 2.3x speedup in flow analysis.
Wed	26 Aug	Added reset() method clearing all flow aggregates and history for seamless testing resets.
Thu	27 Aug	Created AnomalyDetectionTest.java: implemented unit tests for Shannon entropy calculation and anomalous flow detection.

Day	Date	My Work for the Day
Fri	28 Aug	Executed 'mvn clean test'. Both unit tests passed with 100% success in 0.31 seconds. Pushed Module 3 deliverable to GitHub.
SAT	**29 Aug**	**Mentor meeting. Showed automated unit tests passing and optimized entropy throughput. Mentor approved Module 3 completion.**
Sun	30 Aug	Prepared integration benchmark scenarios combining rule engine and ML anomaly scoring.

MAYANK JANGID — 24E1ARADM30P097

Module: Anomaly & ML Detection Engine (Person 3)

Day	Date	My Work for the Day
WEEK 9 — Integration + Testing		
Mon	31 Aug	Integrated FlowFeatureExtractor and AnomalyDetectionEngine into Nikhil's NsmPipelineCoordinator.
Tue	1 Sep	Verified parallel processing: confirmed ML engine and Rule engine process packets simultaneously with zero lock contention.
Wed	2 Sep	Validated that ML alerts carry purple [🧠 ML MODEL] badges on Parikshit's dashboard alongside rule alerts.
Thu	3 Sep	Tested reset coordination: verified /api/alerts/clear triggers coordinator.resetEngines() and clears flow maps.
Fri	4 Sep	Benchmarked feature extraction latency: computed entropy and flow heuristics in 0.22 ms per packet.
SAT	**5 Sep**	**Mentor meeting. Demonstrated dual detection: both Rule and ML engines firing complementary alerts into SQLite. Mentor commended design.**
Sun	6 Sep	Prepared randomized port scan profiles to test stealth detection on physical LAN.
WEEK 10 — Multi-Host Deployment & Network Simulation		
Mon	7 Sep	Deployed NetSentinel on physical university Wi-Fi subnet for multi-device behavioral testing.
Tue	8 Sep	Monitored ML anomaly scoring during external LAN probes (curl http://192.168.1.5:8088). Anomaly engine flagged abnormal flow with high confidence.
Wed	9 Sep	Executed randomized port scans from secondary laptop; verified Shannon entropy flagged 49 of 50 stealth scans (98% detection rate).
Thu	10 Sep	Streamed 4K video and heavy web traffic on client laptops; confirmed benign traffic maintained H <= 1.12 with zero false alerts.
Fri	11 Sep	Audited flow table memory under continuous multi-client LAN traffic; verified flow pruning evicted stale records reliably.
SAT	**12 Sep**	**Mentor meeting. Presented Shannon entropy live charts and false positive metrics (0.0016% on benign traffic). Mentor signed off Week 10.**
Sun	13 Sep	Documented statistical formulas and benchmark tables for final project report.
WEEK 11 — Performance Profiling, Report + PPT + Video		
Mon	14 Sep	Profiled JVM heap during 6-hour burn-in stress test: GC pause times remained below 4 ms.
Tue	15 Sep	Verified schema compliance for 1-click Export JSON Forensic Audit and Export CSV Threat Report in View 4.
Wed	16 Sep	Authored Module 3 report chapter: Shannon entropy mathematics, flow feature aggregation, and confidence calibration.
Thu	17 Sep	Prepared presentation slides explaining behavioral anomaly inference, entropy calculations, and ML badge tags.
Fri	18 Sep	Recorded 2-minute video demonstrating entropy calculation during stealth port scan. Sent to lead Nikhil Kumar.
SAT	**19 Sep**	**Mentor meeting. Submitted Module 3 report draft and slides. Mentor approved mathematical anomaly descriptions.**
Sun	20 Sep	Refined report draft with mentor recommendations and checked GitHub commit log.
WEEK 12 — Final Submission & Project Viva Completion		
Mon	21 Sep	Verified ML engine execution inside packaged executable Fat JAR (module-4-backend-storage-1.0.0.jar).
Tue	22 Sep	Audited Module 3 code and test files across mirrored Git repositories (verified 100% SHA-256 match).
Wed	23 Sep	Rehearsed viva technical answers on Shannon entropy formula, log2 optimizations, and distinguishing DDoS from legitimate bursts.
Thu	24 Sep	Participated in full team dry-run presentation and live intrusion simulation demonstration.
Fri	25 Sep	Conducted final inspection of project report and presentation slides.
SAT	**26 Sep**	**Mentor meeting. Successfully defended Module 3 before Project Guide Er. Ram Babu Buri. Received outstanding evaluation marks; Module 3 officially signed off!**

Day	Date	My Work for the Day
Sun	**27 Sep**	Academic project archive verification: confirmed final grade records, repository tags, and signed documentation archived.

NIKHIL KUMAR (TEAM LEAD) — 24E1ARADM30P110

Module: Central Backend & SQLite Storage (Person 4 & Project Lead)

Day	Date	My Work for the Day
WEEK 1 — Team Formation + Abstract		
Mon	6 Jul	As Team Leader, registered NetSentinel team on portal under guide Er. Ram Babu Buri. Assigned module ownership to all 5 members. Initialized GitHub repo and root Maven project.
Tue	7 Jul	Analyzed system architecture: selected Modern Java 21 LTS, Spring Boot 3.3.4, Project Loom Virtual Threads, and zero-configuration SQLite storage.
Wed	8 Jul	Defined the 5-stage decoupled processing pipeline: Ingestion -> Rule Engine -> Anomaly Scoring -> SQLite Storage -> Web Dashboard.
Thu	9 Jul	Collected module descriptions from all members and consolidated the master project abstract (about 300 words).
Fri	10 Jul	Pushed initial multi-module Maven structure (parent pom.xml, module folders, .gitignore) to GitHub repository.
SAT	**11 Jul**	**Mentor meeting. Presented NetSentinel concept and architecture. Guide Er. Ram Babu Buri approved project scope and 100% pure Java design.**
Sun	12 Jul	Reviewed mentor recommendations. Established frozen JSON contract specifications for Packet JSON and Alert JSON across module boundaries.
WEEK 2 — SRS Documentation		
Mon	13 Jul	Initiated comprehensive System Requirements Specification (SRS) document according to IEEE standards. Authored Section 1: Purpose & Scope.
Tue	14 Jul	Authored User Roles: Security Analyst (SOC Operator), Network Administrator, and Red-Team Evaluator / Simulated Attacker.
Wed	15 Jul	Authored Database Requirements: SQLite relational engine with Write-Ahead Logging (WAL mode), alerts schema, and indexed source IP queries.
Thu	16 Jul	Authored Non-Functional Requirements: sub-millisecond evaluation latency, memory boundedness, fault isolation, and global CORS compliance.
Fri	17 Jul	Compiled all 5 members' sections into master SRS. Formatted document and generated SRS.html and SRS.pdf.
SAT	**18 Jul**	**Mentor meeting. Presented complete SRS document to guide. Mentor commended thorough non-functional requirements and signed off SRS.**
Sun	19 Jul	Organized SRS deliverables for team reference. Planned UML architectural diagrams.
WEEK 3 — UML Design		
Mon	20 Jul	Designed Master Pipeline Flowchart in Mermaid and draw.io showing packet ingestion, parallel rule/anomaly evaluation, and SQLite commit.
Tue	21 Jul	Designed Database Schema ER Diagram with 'alerts' table (alert_id PK, timestamp, attack_type, severity, src_ip, dst_ip, details, confidence, detection_method).
Wed	22 Jul	Designed Component Sequence Diagram modeling REST API requests (/api/alerts, /api/traffic-stats, /api/summary) from Module 5 frontend.
Thu	23 Jul	Designed Master Class Diagram integrating all 5 modules: coordination services, repositories, and controllers.
Fri	24 Jul	Collected all UML diagrams into central documentation folder and committed updates to GitHub repository.
SAT	**25 Jul**	**Mentor meeting. Presented complete UML diagram suite. Guide approved master architecture and database schema design.**
Sun	26 Jul	Refined database connection pooling configurations for SQLite JDBC and HikariCP.
WEEK 4 — Architecture, DB Schema & Contracts		
Mon	27 Jul	Configured root pom.xml with Maven dependency management for Spring Boot 3.3.4, Jackson 2.17, SQLite JDBC 3.46, and JUnit 5.
Tue	28 Jul	Created module-4-backend-storage with Spring Boot starter dependencies (web, jdbc, test).
Wed	29 Jul	Created application.properties: configured SQLite URL (jdbc:sqlite:nsm.db), Virtual Threads enabled (spring.threads.virtual.enabled=true).

Day	Date	My Work for the Day
Thu	**30 Jul**	Created DatabaseInitializer.java: automated table creation for 'alerts' with indexes (idx_alerts_ts_sev, idx_alerts_src_ip) and WAL mode pragma.
Fri	**31 Jul**	Created NsmBackendApplication.java. Tested running Spring Boot app. Database nsm.db auto-created successfully on startup.
SAT	**1 Aug**	**Mentor meeting. Demonstrated Spring Boot 3 startup and automatic SQLite schema initialization. Mentor approved database layer.**
Sun	**2 Aug**	Planned AlertRepository DAO methods using Spring JdbcTemplate.

NIKHIL KUMAR (TEAM LEAD) — 24E1ARADM30P110

Module: Central Backend & SQLite Storage (Person 4 & Project Lead)

Day	Date	My Work for the Day
WEEK 5 — Module Coding Phase 1		
Mon	3 Aug	Created AlertRepository.java with JdbcTemplate. Implemented save(AlertData alert) with INSERT OR REPLACE logic.
Tue	4 Aug	Implemented findRecent(int limit) querying alerts ordered by timestamp DESC with custom RowMapper.
Wed	5 Aug	Implemented countByAttackType() returning grouped attack counts (port_scan, brute_force, syn_flood) for dashboard counters.
Thu	6 Aug	Implemented findTopSourceIps(int limit) aggregating attack counts per source IP with max(timestamp) last seen.
Fri	7 Aug	Implemented clearAll() executing 'DELETE FROM alerts' for instant session clearing during live demonstrations.
SAT	**8 Aug**	**Mentor meeting. Showed AlertRepository CRUD operations and SQLite queries running in under 2 ms. Mentor approved repository layer.**
Sun	9 Aug	Designed 1Hz rolling traffic statistics buffer for real-time frontend charts.
WEEK 6 — Module Coding Phase 2		
Mon	10 Aug	Created TrafficStatsService.java with @EnableScheduling. Built synchronized 30-point rolling circular buffer.
Tue	11 Aug	Implemented @Scheduled(fixedRate = 1000) recordSecondTick() computing packets-per-second delta and recording timestamps.
Wed	12 Aug	Implemented incrementPacket() and incrementPacketBatch(count) using thread-safe AtomicLong counter.
Thu	13 Aug	Implemented getTrafficStats() returning packets_per_sec list and timestamps list for Chart.js rendering.
Fri	14 Aug	Tested traffic buffer with 500 simulated packets. Verified rolling array updates smoothly without dropping timestamps.
SAT	**15 Aug**	**Mentor meeting. Demonstrated 1Hz rolling traffic rate calculation live. Mentor approved sub-millisecond API response time.**
Sun	16 Aug	Planned inter-module pipeline coordinator linking Modules 1, 2, and 3.
WEEK 7 — Module Coding Phase 3		
Mon	17 Aug	Created NsmPipelineCoordinator.java. Integrated RuleEngine (Module 2) and AnomalyDetectionEngine (Module 3) instances.
Tue	18 Aug	Implemented processIncomingPacket(PacketData packet): dispatches packets to rules and anomaly engine concurrently.
Wed	19 Aug	Configured automated alert persistence: when any detection engine emits an alert, coordinator persists it to AlertRepository.
Thu	20 Aug	Added background benign traffic generator producing realistic web/DNS traffic (10-25 pkts/s) for live baseline activity.
Fri	21 Aug	Added automated decoy probe listener initialization binding ports 2222, 8088, 8888, 9090 on Spring Boot startup.
SAT	**22 Aug**	**Mentor meeting. Demonstrated complete end-to-end pipeline: raw packet -> rule -> ML -> SQLite commit. Mentor signed off Phase 3.**
Sun	23 Aug	Planned REST API controller and attack simulation demonstration triggers.
WEEK 8 — Module Coding Phase 4: Final		
Mon	24 Aug	Created NsmApiController.java exposing frozen contract: GET /api/alerts, GET /api/traffic-stats, GET /api/summary, GET /api/top-ips.
Tue	25 Aug	Configured global CORS (@CrossOrigin(origins = "*")) allowing dashboard to run embedded or standalone.
Wed	26 Aug	Implemented attack simulation demo endpoints: POST /api/attack/port-scan, syn-flood, brute-force, and pcap-upload.
Thu	27 Aug	Implemented POST/DELETE /api/alerts/clear endpoint resetting detection engines, flushing coordinator, and clearing SQLite.
Fri	28 Aug	Wrote NsmBackendIntegrationTest.java using MockMvc. Authored TEAM_LEADER_AND_COLLABORATION_GUIDE.md and established git branch protocol (feature/module-X). Pushed Module 4 deliverable.
SAT	**29 Aug**	**Mentor meeting. Showed all 4 integration tests passing and REST endpoints responding in under 6 ms. Mentor confirmed Module 4 complete.**

Day	Date	My Work for the Day
Sun	**30 Aug**	Audited multi-module build reactor and prepared cross-module integration tests.

NIKHIL KUMAR (TEAM LEAD) — 24E1ARADM30P110

Module: Central Backend & SQLite Storage (Person 4 & Project Lead)

Day	Date	My Work for the Day
WEEK 9 — Integration + Testing		
Mon	31 Aug	Executed full cross-module integration test across all 5 modules. Verified end-to-end packet processing lifecycle from capture to SQLite commit.
Tue	1 Sep	Audited SQLite database concurrency under WAL mode (journal_mode=WAL): tested simultaneous alert writes from rule and ML engines with zero lock contention (SQLITE_BUSY = 0).
Wed	2 Sep	Embedded Module 5 static frontend assets into module-4/src/main/resources/static/ with cache-busting version query parameters (?v=2).
Thu	3 Sep	Fixed backend reset synchronization: ensured /api/alerts/clear executes atomic SQLite transaction (DELETE FROM alerts) and resets in-memory ring buffers and coordinators.
Fri	4 Sep	Executed parent project build 'mvn test'. All 13 unit and integration tests passed cleanly in 15.3 seconds.
SAT	**5 Sep**	**Mentor meeting. Demonstrated full 5-module integrated platform running end-to-end. Guide commended clean integration and signed off Week 9.**
Sun	6 Sep	Prepared multi-machine deployment environment and LAN router configurations for physical network penetration tests.
WEEK 10 — Multi-Host Deployment & Network Simulation		
Mon	7 Sep	Deployed NetSentinel on primary laptop over physical university Wi-Fi subnet (192.168.1.x). Configured subnet IP binding (0.0.0.0:8080) for multi-device access.
Tue	8 Sep	Monitored live external probes (curl http://192.168.1.5:8088); verified SQLite logged alerts with accurate millisecond timestamps and telemetry.
Wed	9 Sep	Encountered and diagnosed background Java process locking port 8080 (PID 22892); added automated PowerShell port-kill diagnostic commands to runbook and startup scripts.
Thu	10 Sep	Supported loopback normalization testing and verified SQLite correctly stores normalized 127.0.0.1 source IP records.
Fri	11 Sep	Hardened Spring Security Basic Authentication (admin:admin123) and added /api/auth/me identity verification endpoint.
SAT	**12 Sep**	**Mentor meeting. Demonstrated live physical multi-host intrusion detection and SQLite WAL concurrency under load. Mentor signed off Week 10.**
Sun	13 Sep	Organized system benchmarking procedures and Project Loom Virtual Thread profiling.
WEEK 11 — Performance Profiling, Report + PPT + Video		
Mon	14 Sep	Configured Maven Shade / Spring Boot repackage plugin to generate single self-contained executable Fat JAR (module-4-backend-storage-1.0.0.jar) with shaded dependencies.
Tue	15 Sep	Created run.bat one-click batch launcher automating build checks, background cleanup, and instant server startup.
Wed	16 Sep	Synchronized primary repo (network-security-monitoring-platform) and remote mirror (nsm-test-project), running SHA-256 byte audits to verify 100% bit-for-bit parity.
Thu	17 Sep	Compiled master system benchmark matrix: packet parsing (0.08 ms), rule evaluation (0.12 ms), entropy calculation (0.22 ms), SQLite write (1.15 ms), API response (6.4 ms).
Fri	18 Sep	Created Master 20-slide presentation deck and recorded comprehensive live video demonstration.
SAT	**19 Sep**	**Mentor meeting. Presented report draft, benchmark matrix, executable Fat JAR, and PPT deck. Guide provided suggestions for defense viva.**
Sun	20 Sep	Incorporated mentor feedback into presentation deck and final report document.
WEEK 12 — Final Submission & Project Viva Completion		
Mon	21 Sep	Tested fresh clone of project from GitHub repository on clean external computer. Confirmed 1-click startup via run.bat in under 2.2 seconds.
Tue	22 Sep	Verified 100% SHA-256 match across all files between primary repository and mirror repository on GitHub.
Wed	23 Sep	Conducted full team viva question review covering Java 21, Project Loom Virtual Threads, Spring Boot 3, and SQLite WAL mode.

Day	Date	My Work for the Day
Thu	24 Sep	Led full team presentation dry run and live multi-vector attack demonstration dress rehearsal with all 5 members.
Fri	25 Sep	Final inspection of consolidated documentation deliverables and GitHub release assets.
SAT	**26 Sep**	**Mentor meeting. Led team in final project viva defense before Project Guide Er. Ram Babu Buri. Team received outstanding evaluation marks; NetSentinel NSM Platform officially signed off!**
Sun	27 Sep	Academic project archive verification: confirmed final grade records, repository tags, and signed documentation archived.

PARIKSHIT VYAS — 24E1ARADM40P119

Module: SOC Web Dashboard & Frontend (Person 5)

Day	Date	My Work for the Day
WEEK 1 — Team Formation + Abstract		
Mon	6 Jul	Joined NetSentinel team. Selected Module 5 (SOC Web Dashboard & Frontend). Focused on real-time web telemetry and user experience.
Tue	7 Jul	Researched enterprise Security Operations Center (SOC) dashboard designs: dark high-contrast themes, rolling line charts, alert feeds.
Wed	8 Jul	Evaluated frontend visualization libraries: selected Chart.js for lightweight, smooth 60fps canvas rendering without heavy frontend frameworks.
Thu	9 Jul	Wrote SOC Dashboard section for abstract (about 250 words) and submitted to lead Nikhil Kumar.
Fri	10 Jul	Created module-5-dashboard folder with index.html, css/style.css, and js/app.js. Pushed initial commit to GitHub.
SAT	**11 Jul**	**Mentor meeting. Presented dashboard layout mockup. Mentor recommended adding Live/Pause controls for evaluator demonstrations.**
Sun	12 Jul	Reviewed mentor feedback. Planned responsive CSS grid layout for top metrics, live telemetry, and bottom alert feed.
WEEK 2 — SRS Documentation		
Mon	13 Jul	Wrote SRS functional requirements for dashboard: 2-second automated REST polling, smooth chart animation, and alert card rendering.
Tue	14 Jul	Wrote requirements for visual alert classification: render severity badges ([HIGH], [MEDIUM], [LOW]) and method tags ([RULE] vs [ML]).
Wed	15 Jul	Wrote requirements for Top Source IPs frequency table and attack summary counters (Port Scan, SYN Flood, Brute Force).
Thu	16 Jul	Wrote requirements for 1-Click Attack Simulation Trigger Bar (Port Scan, SYN Flood, Brute Force, PCAP Replay).
Fri	17 Jul	Integrated Module 5 specifications into team SRS document and pushed updates to GitHub repository.
SAT	**18 Jul**	**Mentor meeting. Showed Module 5 SRS specifications. Guide approved 2-second polling interval and simulation trigger bar.**
Sun	19 Jul	Drafted paper wireframes for top navigation, live status dot, and card components.
WEEK 3 — UML Design		
Mon	20 Jul	Created UML Use Case diagram for SOC Operator interacting with dashboard: view traffic, inspect alerts, toggle pause, trigger attacks.
Tue	21 Jul	Designed Component Architecture diagram showing relationship between HTML DOM, Chart.js canvas, and backend REST endpoints.
Wed	22 Jul	Constructed Sequence Diagram modeling 2-second polling cycle from app.js to /api/traffic-stats, /api/alerts, and /api/summary.
Thu	23 Jul	Constructed Activity Diagram for Live/Pause toggle freezing DOM updates while maintaining backend connectivity.
Fri	24 Jul	Exported all UML diagrams to high-resolution PNGs and pushed them to GitHub repository.
SAT	**25 Jul**	**Mentor meeting. Presented dashboard UML diagrams. Guide approved component hierarchy and polling sequence.**
Sun	26 Jul	Researched Chart.js streaming performance and Canvas 2D optimization techniques.
WEEK 4 — Architecture, DB Schema & Contracts		
Mon	27 Jul	Created index.html wireframe: top header, controls bar, attack simulation bar, telemetry grid, alerts list, and top IPs table.
Tue	28 Jul	Authored style.css: established dark cybersecurity color palette with CSS custom properties, cards, and responsive grids.
Wed	29 Jul	Set up app.js controller structure: configured API_BASE URL detection for localhost:8080.
Thu	30 Jul	Added Google Fonts (Inter and JetBrains Mono) for modern typography across telemetry and code metrics.
Fri	31 Jul	Tested opening index.html in Chrome and Edge. Layout rendered cleanly with zero CSS console errors.

Day	Date	My Work for the Day
SAT	**1 Aug**	**Mentor meeting. Showed initial HTML/CSS mockup rendered in browser. Mentor approved dark command center aesthetic.**
Sun	2 Aug	Planned Chart.js integration and real-time polling loops in JavaScript.

PARIKSHIT VYAS — 24E1ARADM40P119

Module: SOC Web Dashboard & Frontend (Person 5)

Day	Date	My Work for the Day
WEEK 5 — Module Coding Phase 1		
Mon	3 Aug	Integrated Chart.js on ecgCanvas. Initialized real-time packets-per-second line graph with smooth tension curves.
Tue	4 Aug	Implemented startPolling(): set up 2-second setInterval loop fetching data from backend REST endpoints.
Wed	5 Aug	Implemented fetchTrafficStats(): updated Chart.js data.datasets[0].data and timestamps in-place without chart flickering.
Thu	6 Aug	Implemented toggleLiveState(): added Live/Pause button freezing polling when paused and restoring live stream on resume.
Fri	7 Aug	Tested traffic graph with backend TrafficStatsService. Line graph animated smoothly at 60fps across rolling 30-second window.
SAT	**8 Aug**	**Mentor meeting. Demonstrated real-time Chart.js telemetry and Live/Pause toggle working live. Mentor approved chart rendering.**
Sun	9 Aug	Designed Real-Time Security Alerts card components and badge styles.
WEEK 6 — Module Coding Phase 2		
Mon	10 Aug	Implemented fetchAlerts() and renderAlerts(alerts): dynamically renders newest-first security alert cards in alertsContainer.
Tue	11 Aug	Added severity badge styles: [HIGH] in red, [MEDIUM] in amber, and [LOW] in blue with glowing border highlights.
Wed	12 Aug	Implemented detection method badges: blue [⚙️ RULE] for deterministic rules and purple [🧠 ML MODEL] for anomaly scoring.
Thu	13 Aug	Implemented fetchTopIps() and renderTopIps(): built Top Source IPs table displaying attacker IP, alert count, and last seen timestamp.
Fri	14 Aug	Implemented fetchSummary(): updated Port Scan, Brute Force, and SYN Flood counter elements with animated value transitions.
SAT	**15 Aug**	**Mentor meeting. Showed alerts feed with visual method badges and Top IPs table. Mentor commended clear visual distinction of ML alerts.**
Sun	16 Aug	Planned 1-click simulation trigger bar and audio alarm system.
WEEK 7 — Module Coding Phase 3		
Mon	17 Aug	Implemented triggerAttack(type): connects simulation buttons to backend POST /api/attack/port-scan, syn-flood, brute-force.
Tue	18 Aug	Implemented handlePcapUpload(event): built file input trigger uploading .pcap dumps to backend replay endpoint.
Wed	19 Aug	Integrated Web Audio API synthesizer (AudioContext): created dual-tone acoustic alarm siren firing on high-severity alerts.
Thu	20 Aug	Added sound unlock on first user click gesture anywhere on screen to comply with modern browser autoplay policies.
Fri	21 Aug	Tested 1-click attack triggers: clicking 'Trigger Port Scan' immediately produced siren, spiked graph, and generated alert card.
SAT	**22 Aug**	**Mentor meeting. Demonstrated 1-click attack simulation triggers and audio alarm. Guide praised interactive demo capabilities.**
Sun	23 Aug	Planned performance optimizations to eliminate DOM re-rendering jitter during high-frequency alerts.
WEEK 8 — Module Coding Phase 4: Final		
Mon	24 Aug	Implemented DOM fingerprinting (lastAlertsFingerprint): prevents DOM re-rendering when alert IDs have not changed, eliminating jitter.
Tue	25 Aug	Added defensive empty-state placeholder cards when no alerts exist: 'No security alerts detected yet. Listening for traffic...'.
Wed	26 Aug	Tested dashboard across multiple resolutions (1920x1080, 1366x768, tablet viewports). Grid resized smoothly.
Thu	27 Aug	Embedded frontend files (index.html, style.css, app.js) into Module 4's src/main/resources/static/ directory with lead Nikhil.

Day	Date	My Work for the Day
Fri	28 Aug	Pushed final Module 5 code deliverable to GitHub repository with message 'Dashboard complete - v1.0'.
SAT	**29 Aug**	**Mentor meeting. Showed full dashboard operating embedded within Spring Boot. Mentor confirmed Module 5 is complete.**
Sun	30 Aug	Prepared integration checklist with backend and attack generators.

PARIKSHIT VYAS — 24E1ARADM40P119

Module: SOC Web Dashboard & Frontend (Person 5)

Day	Date	My Work for the Day
WEEK 9 — Integration + Testing		
Mon	31 Aug	Tested dashboard integration with live backend. Diagnosed bug where Clear button cleared Top IPs but left Real-Time Alerts cards visible in DOM.
Tue	1 Sep	Identified root cause: clearAlerts() set lastAlertsFingerprint = 'EMPTY' before renderAlerts([]), causing conditional check to fail and skip DOM wipe.
Wed	2 Sep	Re-architected executeClearAlerts(): unconditionally wipes alertsContainer, restores telemetry rhythm, and synchronizes reset state with SQLite backend.
Thu	3 Sep	Engineered high-contrast modal confirmation popup (#clearConfirmModal) with Cancel/Confirm buttons, ESC key listener, and backdrop click handler.
Fri	4 Sep	Built Single-Page Application (SPA) multi-view router (switchView): engineered seamless tab switching between View 1 (Rhythm), View 2 (DPI), View 3 (Simulation), View 4 (MITRE ATTACK), and View 5 (3D Globe).
SAT	**5 Sep**	**Mentor meeting. Demonstrated bug-fixed Clear button, confirmation modal, and SPA multi-view navigation. Mentor commended thorough error diagnosis and signed off Week 9.**
Sun	6 Sep	Reviewed UI navigation ergonomics and planned Mega Navigation Bar hover bridge stabilization.
WEEK 10 — Multi-Host Deployment & Network Simulation		
Mon	7 Sep	Stabilized Mega Navigation Bar dropdowns: engineered invisible CSS hover-bridge pseudoelements (::after) to eliminate premature menu vanishing.
Tue	8 Sep	Developed View 5: Interactive 3D Cyber Threat Globe using Three.js and globe.gl, rendering ballistic attack arcs and pulsing impact beacons.
Wed	9 Sep	Bundled local offline earth textures (earth-night.jpg, earth-dark.jpg) for fully air-gapped / offline lab defense demonstrations without external CDN dependencies.
Thu	10 Sep	Verified 2D Leaflet canvas fallback when WebGL hardware acceleration is unavailable on lab workstations.
Fri	11 Sep	Tested live telemetry from external physical LAN probes; confirmed real-time alert card rendering and audio alarm siren firing.
SAT	**12 Sep**	**Mentor meeting. Demonstrated interactive 3D Threat Globe with ballistic arcs and offline textures. Mentor praised visual sophistication and signed off Week 10.**
Sun	13 Sep	Designed UI improvements for MITRE ATTACK threat vector matrix.
WEEK 11 — Performance Profiling, Report + PPT + Video		
Mon	14 Sep	Corrected 'MITRE ATTACK' naming convention and typography across all UI headings and badges.
Tue	15 Sep	Refactored View 4: Replaced static threat matrix with dynamic active vector filtering (activeVectors = THREAT_SIGNATURES_REGISTRY.filter(hits > 0)), updating live when attacks occur.
Wed	16 Sep	Built clean empty-state banner displaying green armored shield ('ALL SYSTEMS SECURE - 0 ACTIVE THREAT VECTORS DETECTED') when alerts are cleared, eliminating lingering boxes.
Thu	17 Sep	Removed redundant static catalog table and integrated 1-click Export CSV Threat Report and Export JSON Forensic Audit download buttons in coverage header.
Fri	18 Sep	Synchronized updated static frontend assets (index.html, style.css, app.js) into Module 4 static resources. Tagged Frontend-Final-v2.0.
SAT	**19 Sep**	**Mentor meeting. Demonstrated dynamic MITRE ATTACK filtering, green shield empty state, and 1-click report downloads. Mentor gave full approval.**
Sun	20 Sep	Recorded comprehensive 3-minute video demonstrating 3D Threat Globe, attack simulations, dynamic MITRE matrix, and CSV/JSON export.
WEEK 12 — Final Submission & Project Viva Completion		
Mon	21 Sep	Verified zero-lag dashboard execution across Google Chrome, Microsoft Edge, and Mozilla Firefox running against packaged standalone Fat JAR.
Tue	22 Sep	Audited all frontend assets across mirrored Git repositories (verified 100% SHA-256 match).

Day	Date	My Work for the Day
Wed	**23 Sep**	Rehearsed viva technical topics: DOM fingerprinting, WebGL globe rendering, Chart.js canvas optimizations, and dynamic SVG vector generation.
Thu	**24 Sep**	Participated in full team viva rehearsal: operated live forensic drill-down dashboard during simulated multi-vector network attack execution.
Fri	**25 Sep**	Performed final inspection of presentation slides, UI architecture diagrams, and documentation screenshots.
SAT	**26 Sep**	**Mentor meeting. Final project viva defense before Project Guide Er. Ram Babu Buri. Received outstanding evaluation marks for innovative enterprise SOC dashboard; Module 5 officially signed off!**
Sun	**27 Sep**	Academic project archive verification: confirmed final grade records, repository tags, and signed documentation archived.

Evaluation Checklist

NetSentinel NSM Platform • Academic Year 2026-27 • Evaluation Rubric Breakdown

Component	Marks	Deliverable	Responsible
Idea & Abstract	**05**	Abstract Document & Research Scope	Nikhil Kumar (Lead)
SRS / Documentation	**10**	IEEE System Requirements Specification (SRS)	Nandlal Upman & Nikhil Kumar
Design (UML, Master Pipeline, ER)	**10**	System Architecture & UML Diagrams	All Members
Weekly Progress Tracking	**25**	12-Week Detailed Activity Tracking Sheet	Nikhil Kumar (Team Lead)
Implementation & Code Quality	**20**	GitHub Source Repository (5 Java Modules)	All Members
Testing & Attack Simulation	**10**	Automated JUnit Suite & Physical LAN Scans	Piyush Agarwal & Mayank Jangid
Presentation Deck & Video	**10**	20-Slide PPT & Architecture Demo Video	Parikshit Vyas & Nikhil Kumar
Live Demo & Technical Viva	**10**	Live Network Intrusion Demo & Viva Q&A	Each Member
Total	**100**	**Comprehensive Evaluation Benchmark Sign-Off**	

Project Guide: Er. Ram Babu Buri
Department of Computer Science & IT
Specializations: Java, Spring Boot, Network Security, IDS, SQLite

Evaluation Status: Approved & Verified
Date: 26 September 2026
Track: Java PBL A.Y. 2026-27 | 12 Weeks | 72 Operating Days

NetSentinel: Real-Time Network Security Monitoring & Intrusion Detection System • Java PBL A.Y. 2026-27 • 6 July – 26 September 2026 • 12 Weeks • 72 Operating Days